

Squad agêntica: do primeiro comando ao dia a dia

Como montar e operar um time de agentes de IA com o BMAD: instalação, base do projeto e rotina de desenvolvimento.

Anderson Gutierres

Forward Deployed Engineer

Um assistente sozinho escreve código, mas não sustenta um projeto

ONDE UM CHAT GENÉRICO VAI BEM

- Escrever funções isoladas
- Corrigir erros de compilação
- Refatorar trechos de código
- Acelerar protótipos

ONDE ELE SE PERDE

- O contexto some entre sessões
- Decisões de arquitetura mudam a cada prompt
- Sem critério de aceite, sem estratégia de testes
- Ninguém lembra por que algo foi decidido

O problema não é o modelo. É a falta de estrutura em volta dele.

Squad agêntica é um time de papéis, não um chatbot

01

Papéis

Cada agente tem uma responsabilidade e um menu próprio de workflows: analista, PM, arquiteto, UX, dev.

02

Artefatos

Brief, PRD, arquitetura e stories ficam em arquivos versionados. Cada documento vira o contexto da fase seguinte.

03

Gates

Você revisa e aprova antes de avançar. Checagens como o readiness check e o code review seguram a qualidade.

O BMAD-METHOD é uma implementação open source desse modelo. O humano continua decidindo; os agentes executam com contexto.

Seis agentes, cada um invocado como skill

AGENTE	SKILL	FAZ O QUÊ
Mary · Analista	<code>bmad-agent-analyst</code>	Brainstorming, pesquisa, product brief
John · PM	<code>bmad-agent-pm</code>	PRD, épicos e stories, correção de rota
Winston · Arquiteto	<code>bmad-agent-architect</code>	Arquitetura, readiness check
Sally · UX	<code>bmad-agent-ux-designer</code>	Design de experiência
Amelia · Dev	<code>bmad-agent-dev</code>	Sprint, stories, implementação, code review
Paige · Tech Writer	<code>bmad-agent-tech-writer</code>	Documentação e diagramas

Novidade da v6: não existe mais Scrum Master separado; é a Dev quem prepara as stories.

Quatro fases, e cada documento vira contexto da próxima

1 · ANÁLISE · OPCIONAL

Entender o problema

Brainstorming, pesquisa, product brief ou PRFAQ.

2 · PLANEJAMENTO

O que construir

PRD com requisitos e, se houver interface, UX.

3 · SOLUÇÃO

Como construir

Arquitetura, épicos e stories, readiness check.

4 · IMPLEMENTAÇÃO

Construir

Uma story por vez: preparar, implementar, revisar.

QUICK FLOW

Trilha paralela: `bmad-quick-dev` pula as fases 1–3 em trabalhos pequenos e bem entendidos.

01

Primeiros passos

Instalar, perguntar ao `bmad-help` e escolher a trilha certa.

Cinco coisas antes do primeiro comando

ITEM	POR QUÊ
Node.js 20.12+	Exigido pelo instalador, que roda via <code>npx</code>
Git	Artefatos e código viajam juntos no histórico
IDE com agente de IA	Claude Code, Cursor e outras. Para ver todas: <code>npx bmad-method install --list-tools</code>
uv (Python)	Workflows como PRD e arquitetura rodam scripts com <code>uv run</code>
Uma ideia pequena	O primeiro projeto é para aprender o fluxo, não para salvar o trimestre

Um comando instala a squad dentro do seu repositório

```
$ npx bmad-method install
```

- ◆ Diretório → ./api-tarefas
- ◆ Módulos → BMad Method (bmm)
- ◆ Tudo estável? → Sim
- ◆ Ferramenta → claude-code
- ◆ Config → nome, idioma, pasta

```
api-tarefas/  
├ _bmad/          agentes, workflows  
└ _bmad-output/  seus artefatos
```

Na v6, os agentes e workflows são **skills** que a IDE reconhece pelo nome `bmad-*`. No Claude Code, chame com barra: `/bmad-help`.

Travou? O bmad-help lê o projeto e diz o próximo passo

```
> /bmad-help tenho uma ideia de API  
de tarefas, por onde começo?
```

```
→ trilha recomendada  
→ primeira tarefa obrigatória  
→ como segue o resto do processo
```

- Inspeciona o que já foi feito no projeto
- Mostra as opções dos módulos instalados
- Recomenda a próxima tarefa obrigatória
- Roda sozinho ao fim dos workflows de planejamento

Escolha a trilha pelo tamanho do planejamento, não pela pressa

TRILHA	PARA QUÊ	DOCUMENTOS
Quick Flow	Bugs, features simples, escopo claro (1–15 stories)	Só spec técnica
BMad Method	Produtos, plataformas, features complexas (10–50+)	PRD + arquitetura + UX
Enterprise	Compliance, sistemas multi-tenant (30+)	PRD + arquitetura + segurança + DevOps

As contagens de stories servem de orientação, não de regra (docs oficiais). Em projeto existente, comece com `bmad-generate-project-context` para registrar as convenções que já existem.

02

Criar a base

O exemplo do começo ao fim: uma API REST de tarefas com tags e prazos, na trilha BMad Method.

Primeiro o problema, depois os requisitos: brief e PRD

ANALISTA • OPCIONAL

`bmad-product-brief`

Uma API de tarefas para times pequenos, que permita criar, etiquetar e filtrar por prazo.

Saída: brief.md, addendum.md, .memlog.md

PM • OBRIGATÓRIO

`bmad-prd` → Create

Uma descoberta guiada transforma o brief em requisitos testáveis, métricas de sucesso e escopo da v1.

Saída: prd.md, addendum.md, .memlog.md

Revise o escopo no PRD, que é o momento mais barato de mudar. Se houver interface, rode `bmad-ux` em seguida.

A API do exemplo não tem interface, então pulamos esse passo.

Arquitetura: stack, dados, erros e testes decididos uma vez só

ARQUITETO · `bmad-architecture`

- Stack e versões
- Modelo de dados: tarefa, tag, prazo
- Padrões de API e de erros
- Estratégia de testes

Saída: ARCHITECTURE-SPINE.md

RECOMENDADO

`project-context.md`, a constituição do projeto

Regras e convenções que todo workflow lê antes de agir. Você pode gerá-lo ao fim da arquitetura com

`bmad-generate-project-context` ou escrevê-lo à mão.

Na v6, épicos e stories nascem depois da arquitetura

PM · bmad-create-epics-and-stories

Usa o PRD e a arquitetura juntos, então as stories já levam em conta o banco, os padrões de API e a stack.

No exemplo (epics.md real):

Épico 1: Registrar e manter tarefas · 3 stories

Épico 2: Saber o que venceu e o que vence · 2 stories

GATE · bmad-check-implementation-readiness

Verifica se PRD, arquitetura e stories contam a mesma história antes do primeiro commit.

READY

NEEDS WORK

NOT READY

Implemente com READY, ou com ressalvas que você leu e aceitou.

Ao fim da base, o plano inteiro está versionado no Git

```
api-tarefas/  
├ .claude/skills/ ← 46 skills  
├ _bmad/  
└ _bmad-output/  
  ├── planning-artifacts/  
  │   ├── briefs/.../brief.md  
  │   ├── prds/.../prd.md  
  │   ├── architecture/.../ARCHITECTURE-SPINE.md  
  │   └ epics.md  
  ├── implementation-artifacts/ ← fase 4  
  │   ├── sprint-status.yaml  
  │   └ 1-1-criar-listar-e-consultar-tarefas.md  
  └ project-context.md ← opcional; não gerado aqui
```

Os artefatos são texto: dá para revisar em PR, e qualquer agente consegue ler em um chat novo.

O contexto sai da sua cabeça e vai para o repositório.

Estrutura conferida numa instalação real da v6.10 com Claude Code. Os "..." são pastas datadas por execução.

No código existente, mostre à squad como o projeto já funciona

01

Limpe o plano antigo

Arquive PRDs, épicos e stories já concluídos para que nenhum agente trate um plano velho como atual.

02

Gere o contexto do projeto

`bmad-generate-project-context`

Lê o código e registra stack, padrões, nomes e testes no project-context.md.

03

Documente o estado atual

`bmad-document-project`

Se a documentação não existe ou está desatualizada, gera a do estado atual em docs/.

Na dúvida, pergunte: `bmad-help` *tenho uma API existente, por onde começo?* Você pode rodar o document-project a qualquer momento, até no meio do projeto.

No brownfield, o tamanho da mudança define o caminho

MUDANÇA PEQUENA

`bmad-quick-dev`

- Detecta a stack e os padrões do código
- Pergunta se deve seguir as convenções atuais
- Gera a spec, implementa e revisa

MUDANÇA GRANDE

BMad Method, com o rigor necessário

- O PRD parte da documentação existente
- A arquitetura escaneia o código antes de decidir
- UX só se houver tela nova ou padrão novo

Código fora das boas práticas? Você escolhe entre manter as convenções e criar novas. O BMAD oferece a modernização, mas não a impõe.

03

Operar no dia a dia

Uma story por vez. Um chat novo para cada workflow.

O ciclo: preparar, implementar, revisar e repetir

UMA VEZ `bmad-sprint-planning` cria o `sprint-status.yaml` com todos os épicos e stories

1

`bmad-create-story`

Gera o arquivo da story com requisitos, critérios de aceite e contexto.



2

`bmad-dev-story`

Implementa a story de ponta a ponta: código e testes.



3

`bmad-code-review`

Revisão adversarial: aprova ou pede mudanças.

↻ Story aprovada, próxima story

Fim do épico: `bmad-retrospective`

Fora do ciclo: mudança grande, ajuste pequeno e modo autônomo

MUDANÇA GRANDE

`bmad-correct-course`

Mudança significativa no meio do sprint.
O resultado é um plano atualizado ou redirecionado.

AJUSTE PEQUENO

`bmad-quick-dev`

Bug ou ajuste fora do backlog. Vai da intenção à spec e ao código revisado, com poucos checkpoints.

NOVO NA 6.10

`bmad-dev-auto`

Uma iteração sem supervisão, pensada para orquestradores como o módulo opcional bmad-loop.

Cinco erros que viram vibe coding com passos extras

ERRO	CORREÇÃO
Pedir código à Dev sem PRD e arquitetura	Escolha a trilha. O Quick Flow existe para o que é pequeno
Reusar o mesmo chat para vários workflows	Um chat novo por workflow, porque o contexto está nos arquivos
Aprovar artefatos sem ler	Os gates só funcionam se alguém ler o que eles produzem
Stories grandes, misturando épicos	Uma story é uma entrega pequena e testável
Pular o readiness check	Rode <code>bmad-check-implementation-readiness</code> antes do sprint

04

Na prática: do PRD à retrospectiva

A API de tarefas passou pela squad de verdade. As perguntas do PM são **ilustrativas**; os terminais resumem as saídas reais.

O PRD nasce de uma conversa: o PM pergunta antes de escrever

> bmad-prd API de tarefas para times pequenos, com tags e prazos.

PM ► Quem usa e o que dói hoje?

> Devs que perdem prazos em planilha.

PM ► O filtro por prazo usa o fuso do usuário ou do servidor?

prd.md real · resumido

3. Glossário

Hoje: data corrente em America/Sao_Paulo

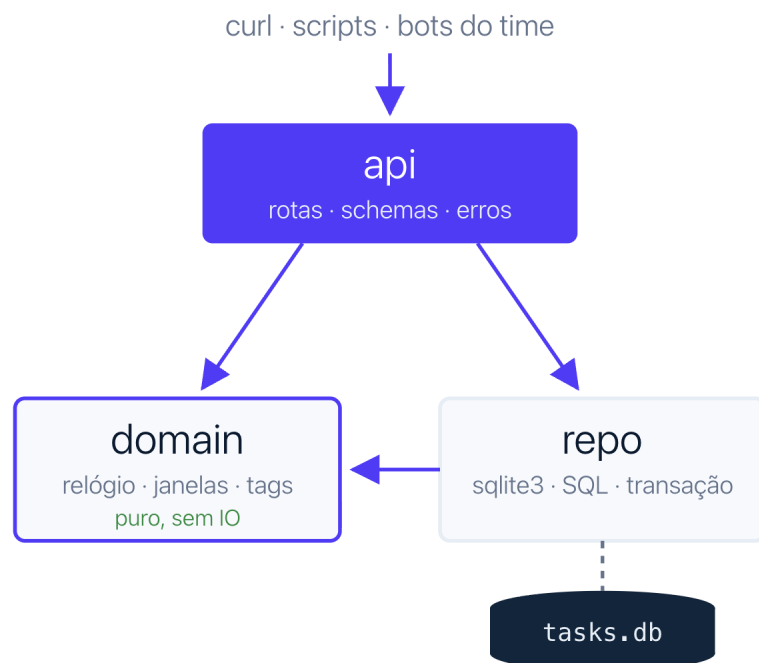
4. Funcionalidades

FR-1 Criar tarefa	FR-5 Etiquetar
FR-2 Listar/consultar	FR-6 Filtrar tag
FR-3 Editar	FR-7 Janela prazo
FR-4 Excluir	FR-8 Tag + janela

6.2 Fora

- Usuários, autenticação e permissões
- Paginação (vai para o backlog)

A arquitetura fixa o que dois agentes decidiriam diferente



Setas = quem pode depender de quem. Nenhuma outra dependência é permitida.

Python 3.14

FastAPI 0.142

sqlite3

pytest

uv

AD-2

Um único relógio

"Hoje" só sai de `domain.today()`, no fuso de São Paulo. `date.today()` é proibido.

AD-7

Um formato de erro

`{"error": {code, field, message}}` em toda resposta de erro.

O readiness check deu PASS, mas apontou onde a story vai doer

- bmad-check-implementation-readiness
- Cobertura: 8/8 FRs · 5/5 NFRs
- Críticos: 0
- M1 Story 1.1 grande (12 critérios)
 - validar com create-story (VS)
- m3 FR1 diz "ISO 8601"; as stories só aceitam YYYY-MM-DD
- Resultado: PASS com ressalvas

O QUE O GATE ACHOU

PASS

PRD, arquitetura e épicos contam a mesma história: nenhum requisito sem story e nenhuma story dependendo de outra futura.

As ressalvas não bloqueiam: a story 1.1 nasce com notas de arquitetura e armadilhas. Os dois ajustes de texto do PRD eram opcionais e só entraram na retrospectiva.

Story 1.1: a Dev recebe um arquivo, não um histórico de chat

```
1-1-criar-listar-e-consultar-tarefas.md
Status: done
## Story
Como dev do time-piloto, quero criar uma
tarefa com título e prazo e depois
listá-la ou consultá-la.
## Acceptance Criteria (3 de 12)
2. POST /tasks → 201 + tarefa
3. title vazio → 422 validation_error
5. prazo no passado → 201
## Dev Notes · Armadilhas conhecidas
```

- bmad-create-story
- Status: ready-for-dev
- (chat novo) bmad-dev-story
- 12 critérios implementados
- uv run pytest: 24 passed
- ruff check e format: ok
- Status: review

O code review aprovou a story 1.1 e adiou os achados baixos

```
> (chat novo) bmad-code-review
> 12 critérios conferidos
> Achados altos ou médios: nenhum
> Baixos: 2 (ex.: id > 2^63-1 vira 500)
  → adiados no arquivo da story
> Status: done
$ git commit -m "feat: story 1.1 –
  Criar, listar e consultar tarefas"
```

```
sprint-status.yaml real · fim da sprint
development_status:
  epic-1: done
  1-1-criar-listar-e-consultar-tarefas: done
  1-2-editar-e-concluir-tarefa: done
  1-3-excluir-tarefa: done
  epic-2: done
  2-1-etiquetar-tarefas-e-...: done
  2-2-filtrar-por-janela-...: done
```

O review achou e adiou; a retrospectiva deu dono à dívida

```
> bmad-retrospective
> 5/5 stories · 77 testes
> 0 rodadas de correção no review
> Mas: id > 2^63-1 adiado em 3 reviews,
  sem dono
> field "task_id" feria o AD-7: passou em
  2 reviews; o dev da 1.3 achou por acaso
> 5 itens de ação · 3 combinados
$ git commit -m "fix: fecha dívida técnica
  da v1 (itens da retrospectiva)" # 84 passed
```

CAUSA DE SISTEMA

O fluxo só mandava corrigir achados altos e médios. Achado baixo não tinha dono.

Aprovado não é pronto.

Checklist: cinco passos do primeiro dia, três lições da retrospectiva

- ☐ **SETUP**
Node.js 20.12+, Git, uv e a IDE com IA aberta no repositório
- ☐ **ORIENTAR**
`bmad-help` para saber qual trilha seguir e por onde começar
- ☐ **EXECUTAR**
Sprint planning e a primeira story passando pelo ciclo completo
- ☐ **DECIDIR**
Achado que pede decisão vira pergunta, não item adiado
- ☐ **INSTALAR**
`npx bmad-method install` com o módulo BMad Method
- ☐ **PLANEJAR**
PRD e arquitetura lidos e aprovados por você
- ☐ **REVISAR**
"Aprovado" é todos os ADs conferidos, não só "nada alto ou médio"
- ☐ **FECHAR**
Story em done sem nenhum [] aberto, commit feito e a próxima na fila

OBRIGADO · PERGUNTAS?

Planejar com agentes é barato. Corrigir sem plano, não.

Anderson Gutierrez

Forward Deployed Engineer

anderson.gutierrez@br.experian.com

DOCS

docs.bmad-method.org

CÓDIGO

github.com/bmad-code-org/BMAD-METHOD

COMUNIDADE

discord.gg/gk8jAdXWmj